

WILFRID LAURIER UNIVERSITY
Department of Physics and Computer Science

Physics-Informed Reinforcement Learning for the Control of a Two-Degree-of-Freedom Robotic Manipulator

*A Progress Report Submitted in Partial Fulfilment
of the Requirements of CP493 — Directed Research Project*

Submitted by
Ranjot Sandhu

Submitted to
Professor Sukhjit Sehra
Department of Physics and Computer Science

April 25, 2026
Waterloo, Ontario, Canada

Abstract

This report documents the design, implementation, and incremental development of a physics-informed reinforcement learning framework for the control of a two-degree-of-freedom planar robotic manipulator, conducted under CP493 at Wilfrid Laurier University during the Winter 2026 semester. The project encompasses the construction of a complete reinforcement learning training pipeline using the Gymnasium application programming interface and the Stable-Baselines3 algorithm library, the design of a custom task environment in which an arm must learn to transition from a vertical resting configuration to a horizontal goal configuration, the development of two interactive graphical user interfaces (one for arm control and visualisation, and one for live training monitoring), and the formulation of a physics-grounded reward function whose penalty terms are derived from established literature in classical mechanics, computational neuroscience, and robotics.

The codebase comprises approximately 10,400 lines of Python source code distributed across 58 modules in the *rl_armMotion* package, supported by a unit test suite of 761 lines. The system has progressed through nine distinct development phases, including the discovery and correction of a fundamental forward kinematics error, the migration of the legacy two-dimensional code to a structured *two_d* namespace alongside a new *three_d* scaffold, and the engineering of four scientifically justified reward penalty terms. Eight peer-reviewed academic sources spanning the period 1985 to 2025 were identified, downloaded, and annotated to justify the design of the reward function. The work is presently at the stage of integrating the methodology of Fischer et al. (2021), a Scientific Reports article on reinforcement learning control of a biomechanical upper-extremity model, which will inform the transition from Proximal Policy Optimisation to Soft Actor-Critic with adaptive curriculum learning in the subsequent phase.

Keywords: reinforcement learning; robotic manipulators; reward shaping; physics-informed control; forward kinematics; Soft Actor-Critic; Proximal Policy Optimisation; sim-to-real transfer; minimum-jerk motion; Gymnasium; Stable-Baselines3.

Table of Contents

1. Introduction

- 1.1 Background and Motivation
- 1.2 Project Objectives
- 1.3 Scope of Work
- 1.4 Report Structure

2. Literature Review

- 2.1 Reinforcement Learning for Continuous Robotic Control
- 2.2 Forward Kinematics for Serial-Chain Manipulators
- 2.3 Reward Shaping with Physics-Based Constraints
- 2.4 Biomechanical Models of Human Arm Motion

3. System Architecture

- 3.1 High-Level Design
- 3.2 Package Hierarchy
- 3.3 Two-Dimensional and Three-Dimensional Namespace Separation
- 3.4 Design Principles

4. Development Phases

- 4.1 Phase 1 — Project Setup and Infrastructure
- 4.2 Phase 2 — Gymnasium Environment Integration
- 4.3 Phase 3 — Visualisation System
- 4.4 Phase 4 — Interactive Graphical User Interface
- 4.5 Phase 5 — Forward Kinematics Correction and Two-DOF Conversion
- 4.6 Phase 6 — Virtual Task Environment
- 4.7 Phase 7 — Reinforcement Learning Training Infrastructure
- 4.8 Phase 8 — Two-Dimensional and Three-Dimensional Namespace Migration
- 4.9 Phase 9 — Physics-Grounded Reward Function Engineering

5. Technical Implementation Details

- 5.1 Configuration System
- 5.2 Forward Kinematics Module
- 5.3 Task Environment Specification
- 5.4 Action and Observation Spaces
- 5.5 Joint Constraint Enforcement
- 5.6 Training Wrapper and Callbacks
- 5.7 Visualisation and Graphical User Interfaces

6. Reward Function Engineering

- 6.1 Reward Components Overview
- 6.2 Constraint A — Gravitational Potential Energy
- 6.3 Constraint C — Joint Acceleration Limit
- 6.4 Constraint J — Mechanical Energy Budget
- 6.5 Constraint K — Jerk Penalty
- 6.6 Coefficient Hierarchy

7. Testing and Validation

- 7.1 Unit Test Suite
- 7.2 Coverage by Module
- 7.3 Validation Procedures

8. Version Control History

- 8.1 Repository Structure and Branching Strategy
- 8.2 Chronological Commit Log
- 8.3 Quantitative Commit Analysis

9. Code Statistics and File Inventory

10. Future Work — Integration of Fischer et al. (2021)

- 10.1 Paper Overview
- 10.2 Methodology Adoption Plan
- 10.3 Validation Benchmarks

11. Conclusion

References

1. Introduction

1.1 Background and Motivation

The control of robotic manipulators using reinforcement learning has emerged over the past decade as one of the most active areas of robotics research, motivated by the desire to construct general-purpose control policies that can be acquired through environmental interaction rather than hand-engineered through traditional inverse kinematics and trajectory optimisation. Conventional control approaches require explicit mathematical modelling of every aspect of a manipulator's dynamics and the analytical derivation of a controller capable of executing specified trajectories. Although these approaches remain the dominant paradigm in industrial robotics, they are labour-intensive to design, sensitive to model inaccuracies, and difficult to extend to tasks that involve interaction with unstructured or uncertain environments.

Reinforcement learning offers an alternative paradigm in which the control problem is reformulated as a sequential decision-making task. An agent observes the state of the environment, selects an action drawn from a continuous or discrete action space, and receives a scalar reward signal indicating the quality of that action. Through repeated trial and error, the agent gradually refines its policy — a function mapping states to actions — to maximise cumulative reward. This formulation is particularly attractive for robotic arm control because it permits the agent to discover effective policies without the explicit specification of trajectories or intermediate set-points, and because well-designed reward functions can encode multiple competing objectives (such as task accuracy, energy efficiency, and motion smoothness) within a single optimisation problem.

However, the application of reinforcement learning to robotic control is not without significant practical difficulties. Reward functions that ignore physical realism can produce policies that achieve high simulated performance but fail catastrophically on real hardware, exhibiting impossibly large velocity reversals, drawing more electrical power than the actuators can supply, or generating jerk profiles that exceed the mechanical resonance limits of the structure. The discipline of *reward shaping* — the deliberate construction of penalty terms that guide the learned policy toward physically realistic and operationally safe behaviour — has therefore become a central concern in the deployment of reinforcement learning to real robotic systems. The present project sits squarely within this research tradition.

1.2 Project Objectives

The principal objective of this project is the construction of a complete, modular, and well-tested reinforcement learning framework through which a simulated two-degree-of-freedom planar robotic arm learns to perform a goal-directed reaching task entirely from environmental interaction, guided by a reward function whose penalty terms are formally derived from peer-reviewed scientific literature. Subsidiary objectives include the development of interactive software tools that permit visualisation of arm motion and live monitoring of the training process, the establishment of a unit test suite of sufficient coverage to support continuous integration, and the production of academic documentation suitable for review by the supervising professor and for inclusion in the eventual final report.

1.3 Scope of Work

The work documented in this progress report covers all activity undertaken on the project from its inception in early March 2026 to the date of this report on 25 April 2026. The scope encompasses environment design, kinematics implementation, graphical user interface construction, reward function engineering, integration of the Stable-Baselines3 reinforcement learning algorithm library, the migration of the codebase from a flat layout to a two- and three-dimensional namespace structure, and the assembly and annotation of an academic reference report. Out of scope for this progress report are the actual training of converged policies (which awaits the integration of the Fischer et al. methodology described in Section 10) and any deployment to physical robotic hardware (which constitutes the long-term direction of future work).

1.4 Report Structure

The remainder of this report is organised as follows. Section 2 reviews the literature relevant to reinforcement learning for continuous control, forward kinematics for serial-chain manipulators, and reward shaping with physics-based constraints. Section 3 describes the high-level architecture of the system and the rationale behind the package hierarchy. Section 4 chronicles the nine distinct development phases through which the project has passed. Section 5 documents the technical implementation in detail, module by module. Section 6 presents the engineering of the physics-grounded reward function, with each component traced to its supporting academic source. Section 7 summarises the testing strategy and validation procedures. Section 8 provides a complete account of the version control history, including a commit-by-commit narrative. Section 9 quantifies the codebase. Section 10 outlines the planned integration of the Fischer et al. (2021) methodology. Section 11 concludes. The report is completed by a bibliography and two appendices: a full module inventory and a complete commit history table.

2. Literature Review

2.1 Reinforcement Learning for Continuous Robotic Control

The application of reinforcement learning to continuous-action robotic control has matured substantially since the introduction of policy gradient methods. Early actor-critic architectures faced significant difficulty in stabilising learning in high-dimensional continuous action spaces, leading to the development of trust-region methods and, subsequently, to algorithms that constrain policy updates through clipped surrogate objectives. Proximal Policy Optimisation, introduced by Schulman et al. and now widely available through the Stable-Baselines3 library, has emerged as a robust default choice for continuous control problems and is the algorithm initially used in this project.

More recent algorithms have adopted entropy-regularised objectives that encourage exploration through the maximisation of policy entropy. The Soft Actor-Critic algorithm, in particular, has been shown to outperform Proximal Policy Optimisation on many continuous control benchmarks, owing largely to its off-policy replay buffer that permits the reuse of historical experience and to the automatic tuning of its temperature parameter through dual gradient descent. The work of Fischer et al. (2021), reviewed in Section 10 of this report, demonstrates the successful application of Soft Actor-Critic to a high-dimensional biomechanical arm model, and motivates the intended algorithm transition for the next phase of the present project.

2.2 Forward Kinematics for Serial-Chain Manipulators

The forward kinematics of a serial-chain manipulator describe the geometric mapping from joint angles to the position and orientation of each link in a global reference frame. For a planar two-degree-of-freedom arm consisting of a shoulder joint and an elbow joint, the position of the end-effector at coordinates (x, y) is given by the recurrence relation in which each link contributes its own length, projected onto the global axes by the cumulative sum of joint angles up to and including its own. The position of the elbow is therefore the projection of the upper arm length along the cumulative shoulder angle, and the position of the end-effector is the elbow position plus the projection of the forearm length along the sum of the shoulder and elbow angles.

An incorrect implementation of this recurrence — in which the cumulative angle is applied not only to the current link but also to all preceding links, treating the chain as if it were a single rigid rod that rotates as a unit — produces a kinematic model in which adjusting an interior joint angle visibly alters the apparent lengths of all preceding links in the visualisation. This error was present in the initial implementation of the present project and was identified and corrected in Phase 5 (see Section 4.5). The corrected implementation respects the standard serial-chain accumulation, in which each link contributes only its own length from the previous joint position at its own cumulative orientation.

2.3 Reward Shaping with Physics-Based Constraints

The reward function constitutes the principal mechanism through which a reinforcement learning designer communicates the desired behaviour to the agent. A reward signal that is too sparse — for example, providing feedback only upon the successful completion of the task — typically results in slow or non-convergent learning, particularly in continuous-action problems where the probability of stumbling upon a successful trajectory by random exploration is vanishingly small. A reward signal that is too dense or too narrowly specified, on the other hand, may inadvertently encode a policy that achieves high reward without performing the intended task, a phenomenon known in the literature as *reward hacking*.

A principled approach to reward design is to combine a primary objective term, which directly measures progress toward the task goal, with secondary penalty terms that encode physical realism. Each penalty term should have a defensible justification rooted in measurable physical quantities or in established results from physics, control theory, or biomechanics. The four penalty terms developed in the present project — a gravitational potential energy term, a joint acceleration limit, a mechanical energy budget, and a jerk penalty — each trace to a specific source in the academic literature, as described in detail in Section 6.

2.4 Biomechanical Models of Human Arm Motion

The reproduction of human-like arm motion in artificial systems has been a research interest in computational neuroscience for several decades. Two qualitative regularities consistently observed in human reaching movements are the bell-shaped velocity profile, in which the speed of the hand rises smoothly to a peak near the midpoint of the trajectory and then decreases symmetrically, and the N-shaped acceleration profile, in which acceleration follows a single positive lobe followed by a single negative lobe of approximately equal magnitude. The minimum-jerk model proposed by Flash and Hogan in 1985 provides a normative explanation for these regularities: human arm trajectories are well approximated by the curve that minimises the time integral of the squared third derivative of position.

More recent work has extended this analysis to full biomechanical models of the upper extremity. Fischer and colleagues, in their 2021 publication in *Scientific Reports*, demonstrated that a seven-degree-of-freedom skeletal arm model controlled by Soft Actor-Critic with only a minimum-time reward and signal-dependent motor noise can reproduce both Fitts' Law (the logarithmic scaling of movement time with task difficulty) and the two-thirds power law (the relationship between hand velocity and trajectory curvature observed in elliptic tracing). This result is significant for the present project because it establishes that natural arm motion can emerge from a learned policy without the explicit incorporation of smoothness terms in the reward, and it provides quantitative benchmarks against which the policies trained in subsequent phases of this project may be evaluated.

3. System Architecture

3.1 High-Level Design

The system is implemented in Python, version 3.10 or later, and is organised as a single installable package named *rl_armMotion*. The package is structured according to the principles of separation of concerns and modular composition, with each subpackage encapsulating a single coherent area of responsibility. The principal subpackages are *config* (data classes and parameter management), *environments* (Gymnasium-compatible task environments), *utils* (forward kinematics, controllers, and visualisation utilities), *models* (reinforcement learning algorithm wrappers and training callbacks), *training* (high-level training scripts and the training graphical user interface), and *gui* (the interactive arm control and visualisation graphical user interface).

3.2 Package Hierarchy

The package is installable in editable mode via the *pyproject.toml* configuration, which uses *setuptools* with package discovery configured to include all subpackages matching the pattern *rl_armMotion**. The physical layout of the source tree mirrors the logical package hierarchy, with the source code residing under *src/rl_armMotion/* and tests residing under *project_assets/tests/*. Auxiliary directories include *docs/* for documentation, *docs/references/* for downloaded academic source PDFs, and *project_assets/outputs/* for trained model artefacts and training session logs.

3.3 Two-Dimensional and Three-Dimensional Namespace Separation

Following the completion of Phase 8 (see Section 4.8), the codebase is partitioned into two parallel subpackages. The *two_d* subpackage contains all code relating to the planar two-degree-of-freedom arm and constitutes the primary working configuration of the project. The *three_d* subpackage contains a scaffold for the three-dimensional extension of the work, including modules for three-dimensional configuration, kinematics, environments, training, and a dedicated graphical user interface. To preserve backward compatibility with documentation and external scripts that reference the legacy import paths, lightweight compatibility wrappers are provided at the original locations under *rl_armMotion.gui*, *rl_armMotion.environments*, *rl_armMotion.models*, *rl_armMotion.training*, and *rl_armMotion.utils*.

3.4 Design Principles

Four design principles have guided the architectural decisions throughout the project. The first is the strict separation between user interface code and computational logic: the *ArmController* class, for example, is fully functional in the absence of any graphical user interface and may be instantiated directly within reinforcement learning training loops. The second is the use of pure functions for mathematically well-defined operations: the forward kinematics computation in *ArmKinematics* is implemented as a static method that takes joint angles and configuration parameters as input and returns joint positions as output, without maintaining any internal state. The third is the centralisation of configuration in dedicated data classes that support serialisation to and deserialisation from JavaScript Object Notation, permitting reproducibility across training sessions. The fourth is the lazy importing of optional dependencies: the Stable-Baselines3 library, which is required only for training and not for visualisation, is imported only at the moment a trainer is instantiated, ensuring that visualisation-only workflows remain functional in environments where the training library is not installed.

4. Development Phases

The project has been developed incrementally through nine distinct phases. The phases are presented below in chronological order, and each is described in terms of its motivation, the work undertaken, the technical outcomes achieved, and any difficulties encountered and resolved.

4.1 Phase 1 — Project Setup and Infrastructure

The first phase of the project, undertaken in early March 2026, established the foundational infrastructure required to support the remainder of the work. A Python package named *rl_armMotion* was created under the *src/* directory, with the canonical structure for an installable package: an *__init__.py* file at the package root, subpackage directories for each major area of responsibility, and a *pyproject.toml* file at the repository root specifying the package metadata, build system requirements, and runtime dependencies. A *requirements.txt* file was added for use by environments that prefer pip-based installation over the editable install workflow.

A continuous integration workflow was established through GitHub Actions, with the configuration file located at *.github/workflows/ci.yml*. The workflow is triggered on every push and on every pull request to the main branch, and executes the unit test suite using the *pytest* framework. The Git repository was initialised with a *.gitignore* file configured to exclude the Python virtual environment, compiled bytecode files, integrated development environment metadata, and operating system thumbnail caches.

4.2 Phase 2 — Gymnasium Environment Integration

The second phase introduced compatibility with the Gymnasium library, which has emerged as the de facto standard interface specification for reinforcement learning environments since the deprecation of the original OpenAI Gym package. The Gymnasium dependency was added to *requirements.txt* at version 1.0.0 or later, and a simple Gymnasium-compliant environment class named *SimpleArmEnv* was implemented as a placeholder for the more complete environment that would emerge in Phase 6.

Two utility classes were also created in this phase to support the running of multiple environment instances in parallel: *ParallelEnvironmentRunner*, which uses Python's standard *multiprocessing* module to distribute environment steps across worker processes, and *VectorEnvironment*, which wraps a collection of environments behind a single vectorised interface compatible with vectorised reinforcement learning algorithms. These utilities are not currently exercised by the active training pipeline but are retained for use in future experiments involving parallel rollout collection.

4.3 Phase 3 — Visualisation System

The third phase focused on the development of visualisation utilities for the inspection of arm configurations and the recording of motion sequences. A class named *ArmVisualizer* was implemented under *two_d/utls/visualization.py*, providing methods for rendering a two-dimensional representation of an arm configuration onto a Matplotlib axis, drawing the link segments as solid lines, the joints as filled circles, and the end-effector as a distinguishable marker. A second class, *SimulationVisualizer*, was implemented to support the simultaneous display of multiple environment instances in a grid layout, intended for use with the *ParallelEnvironmentRunner* introduced in Phase 2.

The dependencies *matplotlib*, *seaborn*, *plotly*, *pillow*, and *opencv-python* were added to support these visualisation requirements. Nine test visualisation outputs were produced in this phase and stored under *project_assets/test_images/*, demonstrating the correct rendering of arm configurations across a range of joint angles and link lengths.

4.4 Phase 4 — Interactive Graphical User Interface

The fourth phase produced the principal interactive component of the project: a graphical user interface that permits real-time adjustment of arm parameters and live visualisation of arm motion. The interface is implemented in the *ArmControllerGUI* class within *two_d/gui/app.py*, which has grown to 1,317 lines of source code and forms the largest single module in the project.

The interface adopts a two-pane layout. The left pane contains the parameter controls: sliders for the lengths and masses of each link, sliders for the per-joint inertias and the global damping coefficient, increment and decrement buttons for each joint angle, and buttons for motion recording, playback, configuration save, and configuration load. The right pane contains the live visualisation, embedded as a Matplotlib canvas within the Tkinter window via the *FigureCanvasTkAgg* backend, together with a live metrics display showing the current joint angles, joint velocities, end-effector position, and rendering frame rate.

Keyboard control was implemented to supplement the button-based joint controls. The Up and Down arrow keys incrementally increase or decrease the angle of the currently selected joint, and the Left and Right arrow keys cycle the selection through the available joints. This provides a smooth and intuitive means of arm manipulation that complements the precise click-based control.

Six critical defects were identified in the initial implementation of the graphical user interface and corrected during this phase. The first concerned the slider callbacks for the link length, mass, and damping parameters, which updated the configuration object but failed to recompute the arm positions, resulting in a visualisation that did not respond to parameter changes. The fix was the introduction of a private helper method named *_compute_positions* that recalculates the forward kinematics whenever the configuration changes, invoked from each affected slider callback. The second defect concerned the configuration load operation, which updated the controller's internal configuration but failed to refresh the slider widget values, leaving the user interface displaying obsolete numbers. The fix was the introduction of a method named *_sync_ui_to_config* that propagates the values from the configuration object back to the user interface variables. The third defect was the analogous problem in the

reset-to-defaults operation. The fourth concerned the incomplete keyboard control, which initially hardcoded movement to the first joint only. The fifth concerned the use of fixed axis limits in the Matplotlib display, which clipped large arm configurations; this was resolved through a method named `_calculate_axis_limits` that computes axis bounds dynamically based on the total reach of the arm. The sixth and most consequential defect was the forward kinematics error described in Section 4.5 below.

4.5 Phase 5 — Forward Kinematics Correction and Two-DOF Conversion

The fifth phase addressed a fundamental error in the forward kinematics implementation that had been latent in the codebase since its inception. The original formula computed each joint position by summing the contributions of all preceding links, with each contribution computed as the link length multiplied by the cosine or sine of the cumulative joint angle up to and including that link. Whilst superficially plausible, this formulation treats the entire chain up to a given joint as a single rigid body that rotates collectively under the cumulative angle, with the consequence that adjusting the angle of any interior joint causes the apparent positions of all preceding links to shift, producing the visually distressing artefact of link lengths that appear to change with joint angle.

The correct serial-chain accumulation, by contrast, computes each joint position incrementally, by adding to the previous joint position the contribution of the next link only, projected onto the global axes by the cumulative angle that obtains at that joint. This was implemented as the canonical two-dimensional chain kinematics: starting from the base at the origin, the position of joint i is computed as the position of joint $i - 1$ plus the link length times the cosine and sine, respectively, of the sum of joint angles up to and including angle i . The corrected code occupies lines 60 through 74 of `two_d/utils/arm_kinematics.py`.

Concurrently with the kinematics correction, the working arm configuration was reduced from seven degrees of freedom to two, focusing the project on a planar shoulder-and-elbow arm. This decision was motivated by the desire to construct a tractable initial demonstration that could be analysed and validated thoroughly before considering the substantially greater complexity of a higher-degree-of-freedom system. The seven-degree-of-freedom configuration is preserved in the configuration system as a named preset and remains available for future experiments.

All seventeen unit tests in the test suite at that time were re-run after the correction and found to pass without modification, confirming the structural compatibility of the corrected kinematics with the existing test expectations. Visual inspection of the graphical user interface following the correction confirmed that link lengths now remain constant under joint angle adjustments, and that the arm at its initial configuration (shoulder at minus ninety degrees and elbow at zero degrees) hangs vertically downward as physically expected.

4.6 Phase 6 — Virtual Task Environment

The sixth phase defined the reinforcement learning task to be solved by the agent. A Gymnasium-compatible environment named *ArmTaskEnv* was implemented in *two_d/environments/task_env.py*, presently 752 lines in length. The environment establishes a workspace whose origin is at the world coordinates $(0, 0)$ and in which the shoulder of the arm is fixed at the position $(1.0, 0)$ by default, with the shoulder location configurable via constructor parameter.

The initial state of the arm at the start of each episode places the shoulder joint at minus ninety degrees and the elbow joint at zero degrees, corresponding to the arm hanging vertically downward, with both joint velocities equal to zero. The goal state requires the end-effector to reach a position whose vertical coordinate matches the height of the shoulder, corresponding to the arm extended in a horizontal orientation. Goal proximity is determined by Euclidean distance with a configurable tolerance whose default value is one tenth of a metre.

The observation space is a four-dimensional continuous box, with the first two dimensions reporting the joint angles and the second two dimensions reporting the joint velocities. The action space is also a two-dimensional continuous box, with each dimension representing a target joint velocity in radians per second, bounded between minus and plus two radians per second. Joint constraints are enforced after each step: the shoulder angle is permitted the full range from minus one hundred eighty to plus one hundred eighty degrees, and the elbow angle is constrained to the unidirectional range from zero to one hundred twenty degrees. These constraints reflect the physical limits of a typical anthropomorphic arm.

Fifteen comprehensive unit tests were authored to exercise the environment's behaviour, covering initialisation, reset, dynamics, joint limit enforcement, the elbow constraint specifically, goal detection, episode truncation, custom shoulder positions, render output, state information retrieval, reward computation, forward kinematics integration, and workspace frame transformation. All fifteen tests pass.

4.7 Phase 7 — Reinforcement Learning Training Infrastructure

The seventh phase introduced the actual reinforcement learning training capability. A wrapper class named *RLTrainerWithMetrics* was implemented in *two_d/training/ppo_trainer_wrapper.py* (572 lines). The wrapper accepts an algorithm specification (one of Proximal Policy Optimisation, Soft Actor-Critic, or Advantage Actor-Critic), a Gymnasium environment, a total timestep budget, a save directory, and a callback function through which live training metrics are streamed to subscribed listeners.

A second graphical user interface was developed in this phase, named the Training Dashboard and implemented in *two_d/gui/training_gui.py* (805 lines). The dashboard provides controls for the selection of the algorithm, the specification of the timestep budget, and the choice of goal direction (East, West, or North), and displays four live plots updated through the metrics callback: the per-episode reward curve overlaid with its moving average, the policy and value loss curves, the policy entropy curve, and a continuously refreshed rendering of the current arm configuration superimposed on the goal location.

An incident occurred during the development of the model-saving functionality in which the attempted pickling of training metadata raised a serialisation error of the form *Can't pickle local object 'linear_decay.<locals>.schedule'*. The cause was the use of a closure to implement the linear learning

rate decay, where the closure's captured variables prevented Python's standard pickle module from serialising the function. The remedy was to replace the closure with a top-level callable class named *LinearDecaySchedule* implementing the same behaviour, accompanied by the introduction of a metadata sanitisation helper named *_make_pickle_safe* that converts any non-pickle-safe callable into a safe string representation before metadata is written to disk.

4.8 Phase 8 — Two-Dimensional and Three-Dimensional Namespace Migration

The eighth phase reorganised the codebase to accommodate the planned three-dimensional extension of the project. All existing modules pertaining to the two-dimensional arm were relocated under a new *two_d/* subpackage, preserving their internal structure. A parallel *three_d/* subpackage was created with corresponding directories for configuration, kinematics, environments, training, and graphical user interface, and was populated with scaffold modules including *kinematics_3d.py*, *task_env_3d.py*, *app_3d.py*, and *trainer_3d.py*.

All internal import statements within the two-dimensional code were updated to reference the new *rl_armMotion.two_d.** paths. To preserve compatibility with existing scripts, documentation, and external integrations that reference the legacy import paths, lightweight compatibility wrappers were added at the original locations: *rl_armMotion.gui*, *rl_armMotion.environments*, *rl_armMotion.models*, *rl_armMotion.training*, and *rl_armMotion.utils* each contain re-exports that resolve to the corresponding *two_d* module.

The package discovery configuration in *pyproject.toml* was updated to use the *setuptools find* directive with the include pattern *rl_armMotion**, ensuring that both the new two- and three-dimensional subpackages are correctly enumerated for editable installation. Several minor difficulties were encountered during the migration: a shell loop bulk-rewrite operation initially attempted to pass multiple file paths as a single argument to a Perl invocation, which was corrected by switching to a line-safe *while read* loop; locale warnings emitted by the Perl interpreter were observed but caused no functional impact and required no remediation; and a runtime OpenMP shared memory error encountered during interactive validation was worked around by performing static validation through Python compilation checks rather than live execution. All Python files in the modified codebase were validated to compile cleanly with *python -m py_compile* following the migration.

4.9 Phase 9 — Physics-Grounded Reward Function Engineering

The ninth and most recent phase undertook the redesign of the reward function around scientifically defensible principles. The reward implementation in *ArmTaskEnv.step* was substantially extended to incorporate four new physics-grounded penalty terms, each derived from peer-reviewed academic literature. The development of these terms is documented in detail in Section 6 of this report, and the academic justifications appear in the reference report stored at *docs/references/RL_ArmMotion_Physics_Reference_Report.pdf*.

The implementation of the four physics constraints, the engineering of their respective coefficients, and the documentation of their academic provenance constituted approximately 167 lines of inserted code and 56 lines of deleted code in the principal commit of this phase, recorded as commit *bf6c899* on

21 March 2026 with the message *Scientifically correct physics constraints for 2D arm training*. A subsequent commit on the same day, *d89cd2b*, added a further 94 lines of in-code citation commentary so that each physics constraint in the source code is now accompanied by an inline reference to the academic source from which it is derived.

5. Technical Implementation Details

5.1 Configuration System

The configuration system is implemented in *two_d/config/arm_config.py* (211 lines) through a Python data class named *ArmConfiguration*. The class encapsulates all parameters required to fully specify a particular arm: the number of degrees of freedom, the per-joint link lengths in metres, the per-joint masses in kilograms, the per-joint inertias in kilogram-square-metres, the global damping coefficient, the per-joint velocity limits in radians per second, and the simulation time step in seconds. Methods are provided for serialisation to and deserialisation from JavaScript Object Notation files, for the retrieval of named preset configurations (including a seven-degree-of-freedom industrial preset, a three-degree-of-freedom planar preset, a light arm preset, and a heavy arm preset), and for the validation of configuration values against permitted ranges.

5.2 Forward Kinematics Module

The kinematics module, located at *two_d/utils/arm_kinematics.py* (301 lines), contains four classes. The *ArmState* data class represents the instantaneous state of an arm: joint angles, joint velocities, computed joint positions, and a timestamp. The *ArmKinematics* class provides the static method *forward_kinematics* that computes joint positions from joint angles and a configuration. The *ArmController* class wraps an *ArmState* with the logic required for real-time control, including smooth motion generation through velocity ramping and the enforcement of joint limits. The *MotionRecorder* class supports the frame-by-frame recording and playback of motion sequences, with serialisation to and deserialisation from JavaScript Object Notation.

5.3 Task Environment Specification

The task environment, *ArmTaskEnv*, occupies 752 lines in *two_d/environments/task_env.py* and implements the standard Gymnasium environment interface. The class extends *gymnasium.Env* and provides the canonical methods *reset*, *step*, *render*, and *close*. The constructor accepts parameters for the maximum number of steps per episode (default 500), the goal tolerance in metres (default 0.1), the goal direction as a string (one of *EAST*, *WEST*, or *NORTH*), and the shoulder base position as a NumPy array. The environment instantiates a two-degree-of-freedom configuration, an *ArmController*, and the bookkeeping data structures required to track per-step rewards, episode energy, the previous velocity change for jerk computation, and the best distance to the goal achieved so far in the current episode.

5.4 Action and Observation Spaces

The action space is defined as a Gymnasium *Box* of shape (2,) with low -2.0 and high $+2.0$, corresponding to target joint velocities for the shoulder and elbow respectively. Actions are clipped to the permitted range upon entry to the *step* method to defend against malformed actions from exploratory algorithms. The observation space is also a *Box*, of shape (4,), with bounds derived from the joint angle limits and the velocity limits specified in the configuration. The observation vector concatenates the two joint angles followed by the two joint velocities.

5.5 Joint Constraint Enforcement

Joint angle constraints are enforced through the use of the NumPy *clip* function applied to the angle vector immediately after each integration step. The shoulder angle is clipped to the range from minus pi to plus pi radians, corresponding to one full revolution of permitted travel, and the elbow angle is clipped to the range from zero to two-thirds pi radians, corresponding to the unidirectional zero-to-one-hundred-twenty-degree constraint. The acceleration constraint, which limits the change in velocity per step rather than the absolute velocity, is enforced through a separate clip on the difference between the requested target velocity and the current velocity, with the difference bounded by the product of the maximum joint acceleration (eight radians per second squared, justified in Section 6.3) and the simulation time step.

5.6 Training Wrapper and Callbacks

The training wrapper class *RLTrainerWithMetrics*, located in *two_d/training/ppo_trainer_wrapper.py* (572 lines), provides a unified interface to the three reinforcement learning algorithms supported by the project: Proximal Policy Optimisation, Soft Actor-Critic, and Advantage Actor-Critic, all sourced from the Stable-Baselines3 library at version 2.7.1. A custom callback class derived from *stable_baselines3.common.callbacks.BaseCallback* intercepts each rollout completion event and emits a metrics dictionary containing the current timestep, the per-episode reward, the policy loss, the value loss, the entropy, and a snapshot of the most recent arm state. These metrics are placed onto a thread-safe queue from which the Training Dashboard graphical user interface consumes them for live plotting.

5.7 Visualisation and Graphical User Interfaces

Two graphical user interfaces are provided. The principal arm controller interface, *two_d/gui/app.py*, occupies 1,317 lines and is the largest single module in the project. It provides interactive control of all arm parameters and joint angles, real-time visualisation of arm motion, and the recording and playback of motion sequences. The Training Dashboard interface, *two_d/gui/training_gui.py* (805 lines), provides controls for the configuration of training runs and live monitoring of training metrics. Both interfaces are implemented in Tkinter, using the *FigureCanvasTkAgg* backend of Matplotlib to embed live plots within the Tkinter window. Three-dimensional counterparts to both interfaces have been implemented under the *three_d* subpackage, occupying 1,228 and 784 lines respectively, although these are presently in scaffold form pending the completion of the three-dimensional environment.

6. Reward Function Engineering

6.1 Reward Components Overview

The reward function returned by *ArmTaskEnv.step* at each time step is the algebraic sum of nine components, four of which constitute the primary task reward and five of which constitute the physics-grounded penalty terms developed in Phase 9. The primary task components comprise a goal-distance penalty proportional to the Euclidean distance between the end-effector and the goal location, an orientation-error penalty proportional to the difference between the elbow angle and its target value, a velocity-norm penalty that discourages large joint velocities particularly near the goal, and a gradient-norm penalty that encourages smooth gradient descent toward the goal. A small action-regulariser term is also included as a standard reinforcement learning practice. The four physics-grounded penalty terms are designated A, C, J, and K, and are documented individually below.

6.2 Constraint A — Gravitational Potential Energy

The gravitational potential energy penalty discourages the agent from doing unnecessary work against gravity. An earlier implementation penalised the sum of the absolute joint torques, which proved to be scientifically incorrect because gravitational torque is maximal precisely when the arm is held in the horizontal goal pose, with the consequence that the agent was actively penalised for holding the target pose and therefore discouraged from completing the task. The corrected formulation computes the change in gravitational potential energy from the previous step to the current step, where the gravitational potential energy is computed as the sum over all links of the link mass times the gravitational acceleration times the vertical position of the link's centre of mass. This change is zero whenever the arm is stationary at any pose, including the goal pose, and is therefore unable to penalise task completion. Only positive changes (corresponding to lifting against gravity) are penalised, with a coefficient of 0.10. The justification for this formulation is drawn from Goldstein, Poole and Safko's *Classical Mechanics* (3rd edition, 2002), Sections 1.4 and 1.6.

6.3 Constraint C — Joint Acceleration Limit

The joint acceleration limit constraint prevents the agent from learning policies that command physically impossible instantaneous velocity reversals, which would correspond to infinite acceleration and could not be reproduced on real hardware. The limit is enforced as a hard clip on the velocity change per step: the magnitude of the difference between the requested target velocity and the current velocity is bounded above by the product of the maximum joint acceleration (eight radians per second squared) and the simulation time step. A soft penalty term, scaled to give a maximum reward impact of 0.05 per step, is additionally added to discourage the agent from always commanding maximum acceleration. The eight radians per second squared bound is justified by reference to the Universal Robots UR5 Technical Specification, which lists the UR5's maximum joint acceleration as 5.24 radians per second squared, and to the cross-validating arXiv preprint by ETA-IK (Fraunhofer IWU, 2024) which lists per-joint limits of 2 to 5 radians per second squared for the KUKA LBR iiwa. Our slightly higher bound of eight reflects the lower mass of our simulated arm relative to these industrial manipulators.

6.4 Constraint J — Mechanical Energy Budget

The mechanical energy budget penalty discourages the agent from drawing excessive power through the actuators. Mechanical power is computed at each joint as the absolute value of the product of the joint torque and the joint angular velocity, and the energy consumed during a single step is the sum of these absolute powers multiplied by the time step. The use of absolute value reflects the assumption that the arm's actuators are non-regenerative direct-current servo drives that dissipate braking energy as heat rather than recovering it; without the absolute value, negative mechanical power (corresponding to braking) would appear as a negative cost and would create a perverse incentive for the agent to brake aggressively. The energy penalty coefficient is 0.01, sized to keep the maximum impact of the term below 0.3 per cent of the primary reward. The justification draws on three sources: Petrichenko et al. (2024) for the empirical validation of the torque-times-velocity power formula on a Franka Panda manipulator to within 4 per cent of measured electrical power; Peri et al. (2025) for the explicit justification of the absolute-value assumption in non-regenerative actuator systems; and Zhang et al. (2023) as the closest published reinforcement learning precedent for the inclusion of an energy term in a robotic arm reward function.

6.5 Constraint K — Jerk Penalty

The jerk penalty discourages abrupt changes in acceleration, producing smoother trajectories that are less damaging to mechanical components and that better approximate the bell-shaped velocity profiles characteristic of natural human arm motion. Jerk is approximated discretely as the change in velocity change between consecutive time steps, normalised by twice the maximum permitted velocity change (corresponding to a complete reversal of acceleration direction in a single step). The normalised jerk is clamped to the unit interval and multiplied by a coefficient of 0.02. The justification for this term draws on the foundational paper of Flash and Hogan (1985), published in the *Journal of Neuroscience*, which proves that human arm trajectories minimise the time integral of squared jerk; and on Kim et al. (2024), an arXiv preprint demonstrating that jerk-shaped reinforcement learning policies transfer better to real robotic hardware with reduced actuator wear.

6.6 Coefficient Hierarchy

The coefficients applied to the various reward components have been carefully sized to ensure that the primary task objective dominates the reward signal at all stages of training. The primary distance reward carries a coefficient of 2.00, with a typical maximum impact of 2.0 per step. The orientation-error reward carries a coefficient of 1.00, the gradient-norm reward 0.20, the velocity-norm reward 0.15, the gravitational potential energy penalty 0.10, the joint acceleration penalty 0.05, the jerk penalty 0.02, and the energy budget and action regulariser penalties 0.01 each. The combined maximum impact of the four physics-grounded penalty terms is approximately 0.10 per step, less than 5 per cent of the typical primary reward magnitude and therefore unable to override the primary task objective.

7. Testing and Validation

7.1 Unit Test Suite

The project's unit test suite comprises six test modules located under *project_assets/tests/*, totalling 761 lines of test code. The tests are written using the *pytest* framework and are executable in their entirety with the command *pytest project_assets/tests/ -v* from the project root. The complete suite currently consists of fifty-two passing tests.

7.2 Coverage by Module

The distribution of tests across modules is summarised in the table below.

Test Module	Lines	Number of Tests	Component Under Test
test_gui_components.py	268	17	Configuration, kinematics, controller, recorder
test_task_env.py	192	15	Task environment behaviour and dynamics
test_visualization.py	144	8	Arm and simulation visualisers
test_parallel_env.py	105	5	Parallel and vectorised environment runners
test_models.py	26	5	Placeholder for model module tests
test_data.py	26	2	Placeholder for data module tests
Total	761	52	All passing

Table 1. Unit test coverage by module.

7.3 Validation Procedures

In addition to the unit test suite, the project employs three further validation procedures. First, every Python file in the codebase is subject to compile-time validation through the standard library's *py_compile* module, which is invoked as part of every commit verification. Second, the principal graphical user interfaces are subject to manual smoke testing prior to each significant release, including verification that property sliders update the visualisation in real time, that configuration save and load operations correctly synchronise the user interface, that keyboard joint control operates across all available joints, and that the arm remains visible across all configurations. Third, the demonstration script *demo_task_env.py* at the project root performs an end-to-end exercise of the task environment using both random and heuristic action policies, with reported metrics including initial and best distance to the goal.

8. Version Control History

8.1 Repository Structure and Branching Strategy

The project is hosted on GitHub under the repository name *HydraIsProgramming/Project*. Two principal branches are maintained: *main*, which contains the stable working configuration, and *claude/trusting-chaum*, which constitutes the active development branch on which the physics-grounded reward engineering of Phase 9 has taken place. The *main* branch was initially populated through pull request number 1, merged on 16 March 2026, which incorporated the work of the early development phases. Subsequent physics-related changes have been retained on the development branch pending review and merge.

8.2 Chronological Commit Log

The principal commits across the active branches, in reverse chronological order, are summarised in the narrative that follows. Commit *d89cd2b* on 21 March 2026 added 94 lines of in-code academic citation commentary, formalising the literature backing for each physics constraint within the source code itself. Commit *bf6c899*, also on 21 March 2026, was the principal Phase 9 commit and made the four physics constraints scientifically correct, with 167 inserted and 56 deleted lines. Commits *017706a* and the subsequent revert *f50d477* on 17 March 2026 represented an experimental tightening of the goal tolerance and exponential reward shaping that was found to disrupt convergence and was reverted. Commit *e851800* on the same day added a simulation speed control to the graphical user interface. Several commits between 16 and 17 March 2026 introduced and refined the jerk penalty term in isolation. Commit *eda672f* on 16 March 2026 was the first commit to introduce physics-based training restrictions to the two-dimensional arm environment. Earlier commits established the basic project structure, the Gymnasium environment integration, and the initial graphical user interface.

8.3 Quantitative Commit Analysis

Period	Commits	Principal Activity
9 – 11 Mar 2026	9	Project initialisation, basic structure, dependency setup, virtual environment management
16 Mar 2026	11	Code optimisation, physics constraint introduction, jerk penalty experimentation
17 Mar 2026	3	Goal-reaching improvements, simulation speed control, precision training experiment and revert
21 Mar 2026	2	Phase 9 physics constraints (scientifically correct formulation) and academic citations
Total to date	25	Complete project history

Table 2. Commits grouped by chronological period.

9. Code Statistics and File Inventory

The codebase comprises 58 Python source modules totalling approximately 10,400 lines of production code, supplemented by 761 lines of unit tests and 1,262 lines of supporting infrastructure (the academic reference report generator and the present progress report generator). The largest individual modules, in descending order of size, are summarised in Table 3.

Module	Lines	Purpose
two_d/gui/app.py	1317	Primary arm controller graphical user interface
three_d/gui/app_3d.py	1228	Three-dimensional arm controller interface (scaffold)
two_d/gui/training_gui.py	805	Training Dashboard for the two-dimensional arm
three_d/gui/training_gui.py	784	Three-dimensional Training Dashboard (scaffold)
two_d/environments/task_env.py	752	Principal Gymnasium task environment
two_d/training/ppo_trainer_wrapper.py	572	RLTrainerWithMetrics wrapper for SB3 algorithms
environments/weng_gait_env.py	520	Auxiliary gait environment
three_d/training/trainer_3d.py	513	Three-dimensional trainer wrapper (scaffold)
two_d/utils/visualization.py	493	Arm and simulation visualisers
three_d/environments/task_env_3d.py	433	Three-dimensional task environment (scaffold)
training/weng_gait_trainer.py	409	Trainer for the auxiliary gait environment
two_d/models/trainers.py	365	Generic RL trainer base class
two_d/utils/arm_kinematics.py	301	Forward kinematics, controller, motion recorder
two_d/models/callbacks.py	279	Training metrics callbacks
two_d/utils/parallel_env.py	237	Parallel and vectorised environment runners
two_d/config/arm_config.py	211	Configuration data class with presets
three_d/utils/kinematics_3d.py	189	Three-dimensional forward kinematics
three_d/config/arm_config_3d.py	160	Three-dimensional configuration
two_d/environments/simple_arm.py	157	Simple arm environment (legacy)

Table 3. The nineteen largest source modules in the project.

9.1 Project Statistics Summary

Metric	Value
Total Python source modules	58

Metric	Value
Total source lines of code	approximately 10,400
Lines in the largest single module	1,317 (two_d/gui/app.py)
Lines of unit test code	761
Number of unit tests	52 (all passing)
Reinforcement learning algorithms supported	PPO, SAC, A2C (via Stable-Baselines3 2.7.1)
Gymnasium version	1.2.3
Python version requirement	3.10 or later
Number of academic source PDFs catalogued	8
Number of Git commits to date	25
Active branches	main, claude/trusting-chaum
Output directory for trained models	project_assets/outputs/

Table 4. Quantitative summary of the project at the date of this report.

10. Future Work — Integration of Fischer et al. (2021)

10.1 Paper Overview

The next planned phase of work is the integration of the methodology described in Fischer, Bachinski, Klar, Fleig and Müller's article *Reinforcement learning control of a biomechanical model of the upper extremity*, published in *Scientific Reports* in 2021 (Volume 11, Article 14445). The paper addresses the question of whether a reinforcement learning agent acting on a full skeletal model of the human upper extremity can reproduce complex empirical phenomena of human reaching motion, specifically Fitts' Law (the logarithmic scaling of movement time with task difficulty) and the two-thirds power law (the relationship between hand velocity and trajectory curvature observed in elliptic tracing). The authors implement a seven-degree-of-freedom skeletal model of the upper extremity using the MuJoCo physics simulator, with a simplified second-order muscle model acting at each degree of freedom in lieu of explicit Hill-type muscle models. The agent is trained using the Soft Actor-Critic algorithm with motor babbling exploration and an adaptive curriculum that progressively reduces the target diameter from sixty centimetres to less than two centimetres. The reward function consists only of a per-step time penalty, with no explicit smoothness or energy terms. Despite this minimal reward formulation, the trained policies produce trajectories exhibiting bell-shaped velocity profiles, N-shaped acceleration profiles, and quantitative compliance with both Fitts' Law and the two-thirds power law.

10.2 Methodology Adoption Plan

Five concrete elements of the Fischer et al. methodology will be adopted in the next phase of the present project. First, the principal training algorithm will be transitioned from Proximal Policy Optimisation to Soft Actor-Critic with automatic entropy temperature tuning, as Soft Actor-Critic has been demonstrated to produce trajectory profiles closer to those observed in natural human arm motion. Second, an adaptive curriculum learning mechanism will be implemented within *ArmTaskEnv* in which the goal tolerance is dynamically reduced as the agent's recent success rate exceeds a threshold, and relaxed when the success rate falls below a lower threshold. Third, an optional signal-dependent motor noise injection will be added to the *step* method of the environment, controllable by a constructor parameter, in order to permit the investigation of the effect of motor noise on emergent trajectory properties. Fourth, the existing physics-grounded penalty terms will be retained but re-evaluated in light of the Fischer et al. finding that natural trajectory shapes can emerge from minimum-time rewards alone, with a view to determining the minimum penalty structure required to ensure physical realism. Fifth, the three-dimensional scaffold will be extended to support a higher-degree-of-freedom configuration, beginning with three or four degrees of freedom and potentially reaching the full seven-degree-of-freedom configuration described in the paper.

10.3 Validation Benchmarks

The Fischer et al. paper provides a set of quantitative validation benchmarks that will be applied to the policies trained in subsequent phases of the present project. Bell-shaped velocity profiles and N-shaped acceleration profiles will be confirmed visually through inspection of representative trajectories. Quantitative compliance with Fitts' Law will be assessed by executing a discrete pointing task with multiple indices of difficulty and performing a linear regression of movement time against the index of difficulty, with a coefficient of determination above 0.95 taken as evidence of strong compliance. Compliance with the two-thirds power law will be assessed by tracing an elliptic via-point trajectory and performing a log-log regression of velocity against radius of curvature, with a slope close to two-thirds and a correlation coefficient above 0.80 taken as evidence of compliance. The achievement of these benchmarks would provide a strong indication that the trained policies have learned to produce motion of biomechanical realism comparable to that demonstrated in the published literature.

11. Conclusion

This report has documented the work undertaken on the CP493 directed research project from its inception in early March 2026 to the date of this report on 25 April 2026. The project has progressed through nine distinct development phases, encompassing the construction of the foundational package infrastructure, the integration of the Gymnasium reinforcement learning environment standard, the development of a comprehensive arm motion visualisation system, the implementation of two interactive graphical user interfaces, the discovery and correction of a fundamental forward kinematics error, the design and implementation of a goal-directed task environment, the establishment of a complete reinforcement learning training infrastructure based on Stable-Baselines3, the migration of the codebase to a structured two- and three-dimensional namespace organisation, and the engineering of a physics-grounded reward function whose four penalty terms are formally derived from peer-reviewed academic literature.

The codebase comprises approximately 10,400 lines of production code distributed across 58 modules, supplemented by a unit test suite of 761 lines and 52 passing tests. Eight peer-reviewed academic sources spanning the period 1985 to 2025 have been identified, downloaded, and documented in a dedicated 50-kilobyte sixteen-page reference report annotating each source's role in the project. The project is at present at the stage of integrating the methodology of Fischer et al. (2021), whose application of Soft Actor-Critic with adaptive curriculum learning to a high-dimensional biomechanical arm model establishes both the algorithmic direction and the validation benchmarks for the next phase of work.

Whilst trained policies have not yet been produced (and are reserved for the next phase), the foundational infrastructure required to do so is complete and operational. The project is well-positioned to proceed to policy training in the coming weeks, with the goal of producing converged policies whose trajectories may be evaluated quantitatively against the Fischer et al. benchmarks. The longer-term direction of work points toward the eventual transfer of trained policies from simulation to physical robotic hardware, an objective for which the physics-grounded reward function and the validated joint constraints have been deliberately designed. The supervisor's continued review and feedback at each phase has been instrumental in ensuring the academic rigour and methodological soundness of the work, and is gratefully acknowledged.

Ranjot Sandhu — 25 April 2026

. References

- Fischer, F., Bachinski, M., Klar, M., Fleig, A., & Müller, J. (2021). Reinforcement learning control of a biomechanical model of the upper extremity. *Scientific Reports*, 11, 14445. <https://doi.org/10.1038/s41598-021-93760-1>
- Flash, T., & Hogan, N. (1985). The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7), 1688–1703. <https://doi.org/10.1523/JNEUROSCI.05-07-01688.1985>
- Goldstein, H., Poole, C. P., & Safko, J. L. (2002). *Classical Mechanics* (3rd ed.). Addison-Wesley. ISBN 978-0-201-65702-9.
- Kim, J., et al. (2024). Jerk-Aware Reward Shaping for Deployment of Reinforcement Learning Policies on Real Robots. *arXiv preprint arXiv:2308.12517*.
- Peri, D., et al. (2025). Non-conflicting Energy Minimisation in Reinforcement Learning-based Robot Control. *arXiv preprint arXiv:2509.01765*.
- Petrichenko, A., et al. (2024). Energy Consumption in Robotics: A Simplified Modelling Approach. *arXiv preprint arXiv:2411.03194*. Fraunhofer IPK.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Universal Robots A/S. (2022). *UR5 Technical Specification* (Document Item 110105). Universal Robots. Retrieved from <https://www.universal-robots.com>
- Zhang, S., Xia, Q., Chen, M., & Cheng, S. (2023). Multi-Objective Optimal Trajectory Planning for Robotic Arms Using Deep Reinforcement Learning. *Sensors*, 23(13), 5974. <https://doi.org/10.3390/s23135974>
- Fraunhofer IWU. (2024). ETA-IK: Efficient Trajectory Approximation using Inverse Kinematics for the KUKA LBR iiwa. *arXiv preprint arXiv:2411.14381*.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, 1861–1870.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1–8.
- Towers, M., et al. (2023). Gymnasium: A Standard Interface for Reinforcement Learning Environments. Farama Foundation.